MÁSTER FP IA Y BIG DATA · MÓDULO 5073 · BLOQUE 4

Sesión 3 de 6

Agentes I: el bucle agéntico

Razonamiento y planificación

Cámara de Madrid · Curso 2026-2027 · Martes 18:00–21:30

REPASO · 18:00 · 5 MIN

Del bucle de herramientas al agente

Sesión 2 · el modelo pide, tu código ejecuta

1. La aplicación envía la pregunta y la lista de herramientas.
2. El modelo devuelve una o varias peticiones de llamada con argumentos.
3. Tu código valida los argumentos y ejecuta la función real.
4. El resultado vuelve al modelo, asociado al identificador de la llamada.
5. El modelo pide otra herramienta o redacta la respuesta final.

HOY

Ese bucle se convierte en una clase reutilizable con cuatro mejoras

Herramientas declarativas

Errores devueltos como datos

Aprobación humana

Límite de pasos

ARQUITECTURA OBJETIVO

Dónde estamos



OBJETIVOS DE HOY

Al terminar la sesión serás capaz de…

1. Distinguir llamada simple, workflow y agente, y elegir el nivel de autonomía adecuado para cada problema.
2. Aplicar los patrones de workflow: encadenamiento, enrutamiento, paralelización, orquestador-trabajadores y evaluador-optimizador.
3. Explicar ReAct, la planificación y el papel de los modelos de razonamiento.
4. Programar un agente con condiciones de parada, gestión de errores, trazas y supervisión humana.
5. Reimplementarlo con un framework y valorar qué aporta y qué cuesta.

AGENDA

Tres horas, dos teorías y dos prácticas

Hora	Bloque	Formato
18:00	Repaso: del bucle de herramientas al agente	Charla
18:05	Teoría 1–3: agentes, workflows y razonamiento	Teoría
18:35	P1 agente desde cero (40') · P2 experimentos (20')	Práctica
19:35	Descanso	—
20:05	Teoría 4–6: control, frameworks y sistemas multiagente	Teoría
20:25	P3 el mismo agente con LangChain (40') · P4 enrutador (15')	Práctica
21:20	Preguntas de control	Debate

TEORÍA · 18:05 · APARTADOS 1 A 3

Agentes, workflows y razonamiento

Cuándo dejar que el modelo decida el siguiente paso

1 · QUÉ ES UN AGENTE

Un grado de autonomía, no una categoría

Un agente es un LLM que usa herramientas **en bucle** para alcanzar un objetivo y que **decide el siguiente paso** a partir de lo que observa.

Llamada simple

QUIÉN DECIDE EL FLUJO

Nadie: una petición, una respuesta

Redactar la respuesta a un correo ya clasificado.

Workflow

QUIÉN DECIDE EL FLUJO

Tu código, con pasos fijos y un LLM en algunos

Clasificar el correo; si es un pedido, extraer las líneas y crear un borrador.

Agente

QUIÉN DECIDE EL FLUJO

El modelo, dentro de los límites que fijas

Resolver un correo mixto: identificar al cliente, mirar sus pedidos, abrir una incidencia y contestar.

1 · REGLA DE DISEÑO

El nivel más simple que lo resuelva



Cuestan más

Cada paso del bucle es otra llamada al modelo, con todo el historial.



Tardan más

Varias idas y vueltas antes de la respuesta final.



Son más difíciles de probar

El camino cambia de una ejecución a otra.

Compensan cuando el camino no se puede prever.

2 · PATRONES DE WORKFLOW

Cinco patrones con pasos que fijas tú

Patrón	Idea	En Tostadero Retiro
Encadenamiento	La salida de un paso es la entrada del siguiente, con comprobaciones entre medias	Extraer el pedido, validarlo contra el stock y redactar la confirmación.
Enrutamiento	Clasificar y derivar a un tratamiento especializado	Consultas sencillas a un modelo pequeño; averías al agente técnico.
Paralelización	Varias llamadas independientes a la vez y combinación del resultado	Valorar el mismo correo por urgencia, tono y riesgo de baja del cliente.
Orquestador-trabajadores	Un modelo divide la tarea y reparte subtareas	«Prepara la ruta de visitas de mañana» dividido por distritos.
Evaluador-optimizador	Un modelo genera y otro revisa contra criterios, en bucle	Revisar la respuesta al cliente: tono, datos correctos y ninguna promesa que no podamos cumplir.

Tres formas de organizar el trabajo

ReAct

Yao et al., 2022

Alternar razonamiento, acción y observación. Hoy está implícito en el bucle de herramientas: razona, llama, lee el resultado y vuelve a decidir.

Planificar y ejecutar

Útil en tareas largas

Primero un plan explícito (lista de pasos), después la ejecución, revisando el plan si algo falla.

Reflexión

Revisarse antes de entregar

Revisar el propio resultado antes de darlo por bueno, como un evaluador-optimizador aplicado al agente.

«Piensan» antes de actuar

Menos trucos de prompt: reducen la necesidad de pedir «piensa paso a paso».

Ajusta el esfuerzo a la tarea: esos tokens de razonamiento se pagan.

Desconfía del relato: el razonamiento visible no siempre refleja fielmente lo que hace el modelo.

LA FUENTE FIABLE

La traza de acciones

Lo que el agente hizo de verdad, paso a paso. En s3_agente.py:

```
[paso {paso}] {nombre}({args})
  -> {resultado}
```

PRÁCTICA · 18:35 · 40 MIN

P1 · Un agente desde cero

El bucle de la sesión 2, ahora clase Agente

1. **Herramientas declarativas:** nombre, descripción, esquema, función y si requiere aprobación.
2. **Errores devueltos como datos**, sin romper el bucle.
3. **Aprobación humana** antes de crear_incidencia.
4. **Límite de pasos:** max_pasos=8 por defecto.

```
python s3_agente.py
```

EL CORREO DE EJEMPLO

«Buenas, os escribo del Restaurante Lumbre. La cafetera de dos grupos no calienta y hoy no podemos servir cafés. ¿Además, cuándo llega el pedido pendiente?»

Avería y pedido en el mismo correo: el agente tiene que identificar al cliente, registrar la incidencia y consultar sus pedidos.

Formato Messages de Claude; en clase, anthropic/claude-haiku-4.5 vía OpenRouter (vale cualquier modelo con herramientas)

P1 · S3_AGENTE.PY (SIMPLIFICADO)

El bucle: decide, actúa y observa

```
def ejecutar(self, objetivo: str) -> str:
    mensajes = [{"role": "user", "content": objetivo}]
    for paso in range(1, self.max_pasos + 1):
        r = self.llm.messages.create(..., tools=self._definiciones(), messages=mensajes)
        mensajes.append({"role": "assistant", "content": r.content})
        if r.stop_reason != "tool_use":  # condición de parada: respuesta final
            return "".join(b.text for b in r.content if b.type == "text")
        resultados = []
        for b in (b for b in r.content if b.type == "tool_use"):
            salida, es_error = self._ejecutar(b.name, b.input)
            print(f"[paso {paso}] {b.name}({b.input}) -> {str(salida)[:90]}")
            resultados.append({"type": "tool_result", "tool_use_id": b.id,
                               "is_error": es_error, "content": json.dumps(salida, ...)})
        mensajes.append({"role": "user", "content": resultados})
    return "He alcanzado el límite de pasos sin terminar. Paso el caso a una persona."
```

P1 · S3_AGENTE.PY (SIMPLIFICADO)

Errores y rechazos vuelven al modelo

```
def _ejecutar(self, nombre: str, args: dict):
    h = self.herramientas.get(nombre)
    if h is None:
        return f"La herramienta {nombre} no existe.", True
    if h.requiere_aprobacion:
        if input(f"¿Aprobar {nombre}({args})? ") != "s":
            return ("Un operador humano ha rechazado "
                    "esta acción. No la repitas."), True
    try:
        return h.funcion(**args), False
    except Exception as e:  # args inventados, BD…
        return f"Error al ejecutar {nombre}: {e}", True
```

Devuelve **(resultado, es_error)**: con **True**, el resultado vuelve marcado como error y el bucle sigue.

requiere_aprobacion solo en crear_incidencia: es la acción con efectos.

El rechazo también vuelve al modelo, para que replantee.

s3_agente.py


```
"""Sesión 3. Un agente desde cero: bucle, límite de pasos, trazas y aprobación humana.

Proveedor: Claude (Messages API). El patrón es idéntico con OpenAI o Gemini."""
import json
import os
from dataclasses import dataclass
from typing import Callable

import anthropic
from dotenv import load_dotenv

from herramientas import DEFINICIONES, FUNCIONES

load_dotenv()


@dataclass
class Herramienta:
```

s3_agente.py


```
    nombre: str
    descripcion: str
    esquema: dict
    funcion: Callable
    requiere_aprobacion: bool = False


class Agente:
    def __init__(self, sistema: str, herramientas: list[Herramienta], max_pasos: int = 8):
        self.llm = anthropic.Anthropic()
        self.modelo = os.environ["ANTHROPIC_MODEL"]
        self.sistema, self.max_pasos = sistema, max_pasos
        self.herramientas = {h.nombre: h for h in herramientas}

    def _definiciones(self) -> list[dict]:
        return [{"name": h.nombre, "description": h.descripcion, "input_schema": h.esquema}
                for h in self.herramientas.values()]
```

s3_agente.py


```
    def _ejecutar(self, nombre: str, args: dict) -> tuple[object, bool]:
        """Devuelve (resultado, es_error). Los errores vuelven al modelo para que se corrija."""
        h = self.herramientas.get(nombre)
        if h is None:
            return f"La herramienta {nombre} no existe.", True
        if h.requiere_aprobacion:
            if input(f" ¿Aprobar {nombre}({args})? [s/N] ").strip().lower() != "s":
                return "Un operador humano ha rechazado esta acción. No la repitas.", True
        try:
            return h.funcion(**args), False
        except Exception as e: # argumentos inventados, fallo de BD...
            return f"Error al ejecutar {nombre}: {e}", True

    def ejecutar(self, objetivo: str) -> str:
        mensajes: list = [{"role": "user", "content": objetivo}]
        for paso in range(1, self.max_pasos + 1):
            r = self.llm.messages.create(
                model=self.modelo, max_tokens=2048, system=self.sistema,
```

s3_agente.py


```
        tools=self._definiciones(), messages=mensajes,
    )
    mensajes.append({"role": "assistant", "content": r.content})
    if r.stop_reason != "tool_use": # condición de parada: respuesta final
        return "".join(b.text for b in r.content if b.type == "text")
    resultados = []
    for bloque in r.content:
        if bloque.type == "text" and bloque.text.strip():
            print(f"[paso {paso}] razonamiento visible: {bloque.text.strip()[:100]}")
        if bloque.type != "tool_use":
            continue
        salida, es_error = self._ejecutar(bloque.name, bloque.input)
        print(f"[paso {paso}] {bloque.name}({bloque.input}) -> {str(salida)[:90]}")
        resultados.append({
            "type": "tool_result", "tool_use_id": bloque.id, "is_error": es_error,
            "content": json.dumps(salida, ensure_ascii=False, default=str),
        })
    mensajes.append({"role": "user", "content": resultados})
```

s3_agente.py


```
        return "He alcanzado el límite de pasos sin terminar. Paso el caso a una persona."


SISTEMA = """Eres el agente de atención técnica de Tostadero Retiro S.L.
Objetivo: resolver correos de clientes usando las herramientas.
Procedimiento:
1. Identifica al cliente con buscar_cliente antes de usar su id.
2. Si hay una avería, registra una incidencia con la prioridad adecuada
   (alta si el local no puede servir café).
3. Si preguntan por pedidos, consulta los datos reales; no inventes cifras.
4. Termina con una respuesta breve al cliente, en español y con tono cordial."""

if __name__ == "__main__":
    herramientas = [
        Herramienta(d["name"], d["description"], d["parameters"], FUNCIONES[d["name"]],
                    requiere_aprobacion=(d["name"] == "crear_incidencia"))
        for d in DEFINICIONES
    ]
```

s3_agente.py


```
    agente = Agente(SISTEMA, herramientas)
    correo = ("Buenas, os escribo del Restaurante Lumbre. La cafetera de dos grupos no "
              "calienta y hoy no podemos servir cafés. ¿Además, cuándo llega el pedido pendiente?")
    print(agente.ejecutar(correo))
```

PRÁCTICA · 19:15 · 20 MIN

P2 · Experimentos con el agente

Experimento	Qué observar
Ejecuta el correo de ejemplo	Secuencia de herramientas, prioridad elegida y respuesta final.
Fija max_pasos=2 en el __main__: Agente(SISTEMA, herramientas, max_pasos=2)	Cómo se corta y qué mensaje recibe el cliente.
Rechaza la aprobación	Cómo reacciona el modelo: ¿lo reintenta, lo explica, propone otra cosa?
Quita «Úsala solo si el cliente describe una avería» de crear_incidencia (en DEFINICIONES de herramientas.py) y pregunta «¿Revisáis las máquinas en agosto?»	Si crea incidencias innecesarias.
Quita el paso 1 del procedimiento en SISTEMA	Si se inventa identificadores de cliente.

19:35 · 30 MINUTOS

Descanso

A las 20:05, teoría: cómo controlar al agente, frameworks y varios agentes.

TEORÍA · 20:05 · APARTADOS 4 A 6

Control, frameworks y sistemas multiagente

Límites, trazas y una persona que aprueba lo importante

4 · CÓMO CONTROLAR AL AGENTE

Condiciones de parada

Respuesta final

El modelo contesta sin pedir más herramientas.

Límite de pasos

max_pasos en P1; un límite de llamadas al modelo en LangChain.

Presupuesto

De tokens o de coste.

Tiempo máximo

Un tope de reloj para cada ejecución.

AL CORTAR, EL CLIENTE NO SE QUEDA SIN RESPUESTA

```
"He alcanzado el límite de pasos sin terminar. Paso el caso a una persona."
```

4 · CÓMO CONTROLAR AL AGENTE

Errores, personas, trazas y evaluación

Errores

Vuelven al modelo como resultado marcado como error, sin romper el bucle, y con reintentos limitados.

Supervisión humana

Las acciones con efectos externos o irreversibles requieren aprobación. El rechazo vuelve al modelo.

Trazabilidad

Cada paso: herramienta, argumentos, resultado y tokens. En producción: LangSmith, Langfuse u OpenTelemetry.

Evaluación

Casos verificables («¿se creó la incidencia correcta con la prioridad correcta?») antes que la redacción.

Agencia excesiva



Dar a un agente más herramientas, permisos o autonomía de los que necesita.

REMEDIO



Herramientas mínimas



Permisos mínimos



Aprobación humana para lo que tenga consecuencias

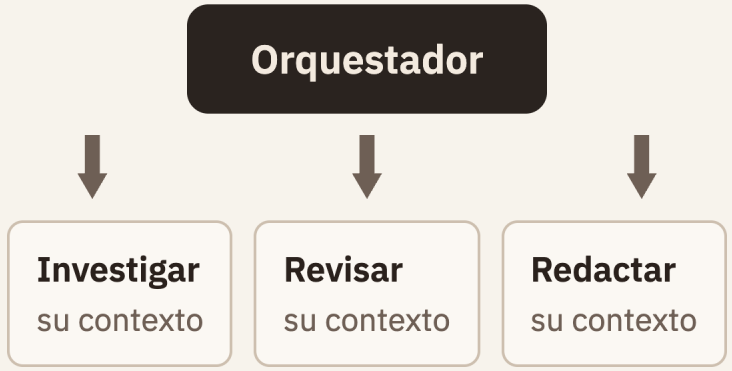
5 · FRAMEWORKS DE AGENTES

El mismo bucle, con otras piezas

Framework	Enfoque	Cuándo tiene sentido
LangChain 1.x / LangGraph	create_agent sobre el motor de LangGraph, con middleware (resumen, aprobación humana, límites) y grafos para flujos a medida	Persistencia, pausas, flujos complejos y varios proveedores. Es el framework del temario.
OpenAI Agents SDK	Agentes, traspasos entre agentes, guardarraíles y sesiones	Proyectos centrados en OpenAI con varios agentes.
Claude Agent SDK	El motor de Claude Code como librería: ficheros, terminal, MCP, subagentes y hooks	Agentes que trabajan sobre ficheros y sistemas, y automatizaciones de desarrollo.
Google ADK	Agentes y multiagente con integración en Google Cloud	Ecosistema Gemini y Vertex.
Pydantic AI	Agentes con tipado fuerte, independientes del proveedor	Equipos que ya trabajan con Pydantic y FastAPI.

Sesión 3 · Agentes I: bucle y razonamiento · Criterio: entiende el bucle a mano antes de adoptar un framework, y fija versiones. En LangChain 1.x, create_agent sustituye a create_react_agent: ojo con los tutoriales antiguos

Un orquestador reparte el trabajo



Ventajas: contexto limpio para cada subagente y paralelismo.

Inconvenientes: muchos más tokens, más latencia y más dificultad para depurar.

Un agente bien diseñado suele rendir más que varios mal coordinados.

PRÁCTICA · 20:25 · 40 MIN

P3 · El mismo agente con LangChain

Con **create_agent**, el bucle lo pone el framework y el control se añade con **middleware**.

La aprobación humana necesita un **checkpointer**: pausar consiste en guardar el estado del grafo y reanudarlo después.

```
python s3_agente_langchain.py
```

Cambio de proveedor: "openai:google/gemini-3.8-flash" usa la misma clave de OpenRouter, porque langchain-openai lee OPENAI_BASE_URL.

Comparad ambas versiones

QUÉ VIENE RESUELTO

Pausa y reanudación, límite de llamadas y cambio de proveedor con una cadena.

QUÉ SE PIERDE

Visibilidad del bucle y control fino del formato de los mensajes.

Y contad las líneas de código de cada versión.

P3 · S3_AGENTE_LANGCHAIN.PY (SIMPLIFICADO)

create_agent y middleware

```
@tool
def buscar_cliente(texto: str) -> list[dict]:
    """Busca clientes por nombre. Devuelve id, nombre, tipo y distrito."""
    return h.buscar_cliente(texto)

agente = create_agent(
    model=f"anthropic:{os.environ['ANTHROPIC_MODEL']}",  # o "openai:google/gemini-3.8-flash"
    tools=[buscar_cliente, consultar_pedidos, crear_incidencia],
    system_prompt=SISTEMA,  # el mismo que en s3_agente.py
    middleware=[
        HumanInTheLoopMiddleware(interrupt_on={"crear_incidencia": True}),
        ModelCallLimitMiddleware(run_limit=8),  # equivale a nuestro max_pasos
    ],
    checkpointer=InMemorySaver(),  # imprescindible para pausar y reanudar
)
```

P3 · LANGGRAPH POR DEBAJO

create_agent construye este grafo



ModelCallLimitMiddleware

El for con max_pasos de P1, como nodo antes del modelo.

HumanInTheLoopMiddleware

El input() de requiere_aprobacion, pero pausando el grafo.

checkpointer

La lista mensajes de P1, guardada tras cada paso del grafo.

P3 · S3_AGENTE_LANGCHAIN.PY

Pausar y reanudar el grafo

```
config = {"configurable": {"thread_id": "correo-lumbre-001"}}
estado = agente.invoke({"messages": [{"role": "user", "content": correo}]}, config)
while "__interrupt__" in estado:  # el grafo se ha pausado esperando a una persona
    decisiones = []
    for peticion in estado["__interrupt__"][0].value["action_requests"]:
        ok = input(f"¿Aprobar {peticion['name']}({peticion['args']})? [s/N] ").lower() == "s"
        decisiones.append({"type": "approve" if ok else "reject"})
    estado = agente.invoke(Command(resume={"decisions": decisiones}), config)
print(estado["messages"][-1].text)
```

thread_id

Identifica el estado guardado por el checkpointer.

__interrupt__

Aparece cuando el grafo se pausa ante crear_incidencia.

Command(resume=…)

Reanuda con una decisión por petición: approve o reject.

s3_agente_langchain.py


```
"""Sesión 3. El mismo agente con LangChain 1.x: create_agent + middleware."""
import os
from typing import Literal

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware, ModelCallLimitMiddleware
from langchain.tools import tool
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

import herramientas as h
from s3_agente import SISTEMA

load_dotenv()


@tool
```

s3_agente_langchain.py


```
def buscar_cliente(texto: str) -> list[dict]:
    """Busca clientes por nombre (coincidencia parcial). Devuelve id, nombre, tipo y distrito."""
    return h.buscar_cliente(texto)


@tool
def consultar_pedidos(cliente_id: int) -> list[dict]:
    """Lista los pedidos de un cliente con fecha, estado y total en euros."""
    return h.consultar_pedidos(cliente_id)


@tool
def crear_incidencia(cliente_id: int, maquina: str, descripcion: str,
                     prioridad: Literal["baja", "media", "alta"]) -> dict:
    """Registra una avería de una máquina de un cliente (maquina = SKU, p. ej. MAQ-010)."""
    return h.crear_incidencia(cliente_id, maquina, descripcion, prioridad)
```

s3_agente_langchain.py


```
agente = create_agent(
    model=f"anthropic:{os.environ['ANTHROPIC_MODEL']}", # o "openai:google/gemini-3.8-flash"
    tools=[buscar_cliente, consultar_pedidos, crear_incidencia],
    system_prompt=SISTEMA,
    middleware=[
        HumanInTheLoopMiddleware(interrupt_on={"crear_incidencia": True}),
        ModelCallLimitMiddleware(run_limit=8), # equivale a nuestro max_pasos
    ],
    checkpointer=InMemorySaver(), # imprescindible para pausar y reanudar
)

if __name__ == "__main__":
    config = {"configurable": {"thread_id": "correo-lumbre-001"}}
    correo = ("Os escribo del Restaurante Lumbre: la cafetera de dos grupos no calienta "
              "y hoy no podemos servir cafés.")
    estado = agente.invoke({"messages": [{"role": "user", "content": correo}]}, config)
    while "__interrupt__" in estado: # el grafo se ha pausado esperando a una persona
        decisiones = []
```

s3_agente_langchain.py


```
        for peticion in estado["__interrupt__"][0].value["action_requests"]:
            ok = input(f"¿Aprobar {peticion['name']}({peticion['args']})? [s/N] ").lower() == "s"
            decisiones.append({"type": "approve" if ok else "reject"})
        estado = agente.invoke(Command(resume={"decisions": decisiones}), config)
    print(estado["messages"][-1].text)
```

PRÁCTICA · 21:05 · 15 MIN

P4 · Un enrutador delante del agente

No todo correo necesita un agente. Combina el clasificador de la sesión 2 con el agente para que **solo los casos complejos usen el camino caro**.

```
from llm_client import LLM                # copia de la sesión 1
from s2_extraer_correo import con_openai  # copia de la sesión 2

SISTEMA_CONSULTAS = "..."                 # lo escribes tú

def atender(correo: str) -> str:
    clasificacion = con_openai(correo)        # sesión 2: salida estructurada
    if clasificacion.tipo == "consulta":      # camino barato y predecible
        return LLM("gemini").responder(SISTEMA_CONSULTAS, correo).texto
    return agente.ejecutar(correo)            # camino agéntico para lo complejo
```

P4 · GUÍA

Qué pones tú y cómo medir

- **Ficheros:** copia llm_client.py (sesión 1) y s2_extraer_correo.py (sesión 2) en esta carpeta; el pack de hoy solo trae crear_bd.py, herramientas.py y los s3_*.
- **Tu parte:** escribe SISTEMA_CONSULTAS y construye agente como en el __main__ de s3_agente.py.
- **Tokens:** agente.ejecutar() solo devuelve el texto, así que suma r.usage dentro del bucle. En el camino barato, la Respuesta de LLM ya trae tokens_entrada y tokens_salida.
- **Modelo barato:** LLM("gemini") va por OpenRouter a google/gemini-3.8-flash, que razona y gasta más tokens de salida. Prueba también LLM("openai", modelo="openai/gpt-5.4-nano") y compara.



Mide con los casos de la sesión 2 cuántas llamadas y tokens ahorra el enrutador.

21:20 · CIERRE

Preguntas de control

1. ¿Qué diferencia un workflow de un agente? Pon un ejemplo de Tostadero Retiro para cada uno.
2. Enumera cuatro condiciones de parada razonables para un agente.
3. ¿Por qué el rechazo de una aprobación se devuelve al modelo en lugar de cortar la ejecución?
4. ¿Qué es la agencia excesiva y cómo se mitiga?
5. ¿Qué te aporta LangChain en este caso y qué te quita?

RECURSOS

Para seguir leyendo

Anthropic, Building effective agents (2024) · patrones de workflow y agentes
anthropic.com/engineering/building-effective-agents

Yao et al., ReAct (2022) · Synergizing Reasoning and Acting in Language Models
arxiv.org/abs/2210.03629

LangChain y LangGraph · agentes, middleware y aprobación humana
docs.langchain.com

OWASP · Top 10 para aplicaciones con LLM y guía para aplicaciones agénticas
genai.owasp.org

OpenRouter · modelos, precios y llamadas a herramientas con los SDK de OpenAI y Anthropic
openrouter.ai/docs

ENTREGABLE E3 · 15 %

Agente de atención técnica desde cero y con LangChain

Con trazas, límite de pasos y aprobación humana: la traza completa de una ejecución con aprobación y de otra con rechazo, y una comparación razonada de una página.

Próxima sesión

Agentes II: memoria y contexto

Requisito: haber completado la sesión 3